

Mathematik für Informatiker 1

Wintersemester 2025/26

Jesse Ratzkin

Universität Würzburg

05 Nov. 2025

Das RSA-Verfahren ist ein Beispiel von Public-Key-Kryptografie, d.h. allgemeine Kryptografie. Wir erklären zuerst die Idee der Public-Key-Kryptografie.

- Alle sollen private Nachrichten miteinander austauschen können.
- Das meint: Jeder braucht eine Methode, um eine Nachricht zu verschlüsseln und Nachrichten zu entschlüsseln.
- Zum Beispiel: Ich muss eine Nachricht für andere verschlüsseln und weiter brauche ich eine Methode, um Nachrichten für mich zu entschlüsseln.

Wie funktioniert Public-Key-Kryptografie in der Praxis?

- Jeder teilt seinen eigenen Verschlüsselungscode mit allen, z.B. in einem großen Buch. Der eigene Entschlüsselungscode wird aber versteckt.
- Wenn A eine private Nachricht zu B sendet, dann findet A den Verschlüsselungscode von B in dem Buch.
- Weil nur B den eigenen Entschlüsselungscode besitzt, kann niemand anders die private Nachricht lesen.
- Der Verschlüsselungscode heißt "Public-Key" und der Entschlüsselungscode heißt "Secret-Key."

Frage: Wenn der Verschlüsselungscode öffentlich ist, kann jeder den Entschlüsselungscode finden?

Praktisch ist eine Nachricht ein Element der Menge $\mathbb{Z}_N$ wobei $N \in \mathbb{N}$ sehr groß ist, und der Verschlüsselungscode ist eine Funktion $f : \mathbb{Z}_N \rightarrow \mathbb{Z}_N$.

Theoretisch kann man die Umkehrfunktion f^{-1} finden, wenn f bekannt ist. Also müssen wir eine Funktion f wählen, die schwer (mit großem Rechenaufwand verbunden) umkehrbar ist.

Wie funktioniert das RSA-Verfahren?

- 1 Wir wählen zwei große Primzahlen p und q und setzen $n = p \cdot q$ und $\phi(n) = (p - 1)(q - 1)$. Eine Nachricht ist ein Element in $\mathbb{Z}_n = \{0, 1, \dots, n - 1\}$.
- 2 Der Public-Key ist eine kleine Zahl e , die teilerfremd zu $\phi(n)$ ist. Wir machen das Paar (n, e) für alle bekannt.
- 3 Der Secret-Key ist das multiplikative Inverse d von e in $\mathbb{Z}_{\phi(n)}$. Dieses findet man durch den Euklidischen Algorithmus.
- 4 Man verschlüsselt eine Nachricht für uns durch die Funktion $f(x) = x^e \bmod n$.
- 5 Wir entschlüsseln unsere Nachrichten durch die Formel $(x^e)^d = x^{e \cdot d} \equiv x \bmod n$.

Wir wählen $p = 23$ und $q = 13$, also ist $n = 299$ und $\phi(n) = 264$. Also ist eine Nachricht eine Zahl in $\mathbb{Z}_{299} = \{0, 1, 2, \dots, 298\}$. Als Public-Key wählen wir $e = 7$. Das ist teilerfremd zu $264 = 2^3 \cdot 3 \cdot 11$. Der Secret-Key ist $d = e^{-1}$ in $\mathbb{Z}_{264}$, was durch $d = 151$ gegeben ist (Sie sollen überprüfen: $3 \cdot 264 - 113 \cdot 7 = 1$ und $113 + 151 = 264$.) Das Paar $(n, e) = (299, 7)$ ist für alle bekannt und die Verschlüsselungsfunktion ist $f(x) = x^7 \bmod 299$.

Wenn Anja die Nachricht $x = 142$ an Birgit sendet, dann meldet sie

$$f(142) = 142^7 = 1,164,175,380,274,048 \equiv 77 \pmod{299}.$$

Anja kann die Nachricht zu Birgit so verschlüsseln. Dannach entschlüsselt Birgit die Nachricht durch die Regel

$$g(y) = g(f(x)) = x^{ed} \pmod{n}.$$

Wir müssen zeigen: Die Entschlüsselungsfunktion $g(y) = y^d$ ist tatsächlich die Umkehrfunktion der Verschlüsselungsfunktion $f(x) = x^e$. Das heißt, wir müssen

$$x = g(f(x)) = x^{ed} \mod n$$

zeigen. Wir wissen schon, dass $n = p \cdot q$, ein Produkt von Primzahlen ist, also reicht es zu zeigen: entweder $x^{ed} \equiv x \mod p$ oder $x^{ed} \equiv x \mod q$. Wir untersuchen den Fall $x^{ed} \equiv x \mod p$ (die gleiche Herleitung gilt für $x^{ed} \equiv x \mod q$).

- Falls x ein Vielfaches von p dann x^{ed} auch ein Vielfaches von p , also folgt $p|(x^{ed} - x)$, d.h. $x^{ed} \equiv x \pmod{p}$.
- Falls x kein Vielfaches von p dann ist $x^{p-1} \equiv 1 \pmod{p}$ (kleiner Satz von Fermat). Aber $ed - 1 = k\phi(n)$ ist ein Vielfaches von $\phi(n) = (p-1)(q-1)$, also folgt

$$x^{ed-1} = (x^{p-1})^{k(q-1)} \equiv 1 \pmod{p} \Rightarrow x^{ed} \equiv x \pmod{p}.$$

Hier ist der gehöriger Python-Code (von Weitz, Kap. 10):

```
from sympy import prime
p, q = prime(50), prime(54)
n=p*q
p, q, n
```

```
from sympy import factorint
phi = (p-1)*(q-1)
print(factorint(phi))
e=7*11*13
e
```

```
from sympy import mod_inverse  
d=mod_inverse(e, phi)  
d
```

```
def crypt (M, k):  
    return M**k % n  
N=51234
```

```
S=crypt (N,e)  
S  
crypt(S,d)
```